CADDack: The ELBO, End to End

What data enters the Bayesian affinity objective, and how it is minimised

CADDack Project

July 20, 2026

Abstract

This note traces the training objective of `FusionAffinityNet` from raw inputs to the scalar loss that is minimised. It shows (i) exactly which tensors enter the model and where, (ii) how they become the predictive parameters (μ, s), (iii) the Evidence Lower Bound (ELBO) and the per-batch loss actually computed in `bayes.py:elbo_loss`, and (iv) the optimisation loop in `train.py:train_fusion_from_complexes`. Every symbol is cross-referenced to the code. The critical scaling detail — the KL is divided by the dataset size N_{train}, not the batch count — is derived in Section 3.3.

§ 1 What data enters, and where

One training example is a protein–ligand complex $\mathcal{C} = (\text{ligand SMILES, 3D geometry, } y)$, built by `gnn/geometry.py:ComplexExample`. It is turned into *two* independent graphs plus a scalar label. The affinity model has two deterministic encoders ("towers") and one Bayesian head.

Symbol	Meaning	Shape	Built by
X_L	ligand atom features	$[n_L, 7]$	`smiles_to_graph_arrays`
$\mathcal{E}_L$, E_L	ligand bonds + bond feats	$[2, m_L]$, $[m_L, 7]$	`smiles_to_graph_arrays`
z	atomic numbers (pocket+ligand)	$[n_G]$	`GeometryRecord`
P	3D coordinates	$[n_G, 3]$	`GeometryRecord`
b_L, b_G	batch-assignment vectors	$[n_L]$, $[n_G]$	PyG `DataLoader`
y	measured affinity (e.g. pK_d)	$[B]$	`geo_batch.y`

Here B is the batch size, n_L / n_G the total ligand / geometry atoms in the batch, and m_L the number of directed ligand edges. The two graphs have *different* atom counts, so they are carried in two parallel PyG batches and zipped together (`train.py:collate_pairs`).

Forward pass: from tensors to (μ, s)

$$\mathbf{h}_L = \textstyle\sum_{\text{pool}} \text{GINE}(X_L, \mathcal{E}_L, E_L, b_L) \qquad \in \mathbb{R}^{B\times H} \qquad \texttt{\_encode\_ligand} \tag{1}$$

$$\mathbf{h}_G = \textstyle\sum_{\text{pool}} \text{Geo}(z, P, b_G) \qquad \in \mathbb{R}^{B\times H} \qquad \texttt{GeometryTower} \tag{2}$$

$$\mathbf{h} = [\,\mathbf{h}_L \,\|\, \mathbf{h}_G\,] \qquad \in \mathbb{R}^{B\times 2H} \qquad \texttt{torch.cat} \tag{3}$$

$$(\mu,\, s) = \text{BayesMLP}(\mathbf{h}) \qquad \mu, s \in \mathbb{R}^{B} \qquad \texttt{bayes.py:BayesianMLP} \tag{4}$$

The head returns two vectors per complex: the predicted mean affinity μ and the *log-variance* $s = \ln \sigma_{\text{al}}^2$ of the observation noise, clamped to $[-10, 10]$ for stability (`bayes.py:135`). The

mean/log-variance heads are ordinary linear layers; only the hidden layers of the head are Bayesian.

§ 2 The Bayesian weights

Each Bayesian hidden layer places an independent Gaussian over every weight (and bias) entry w_{ij} (`bayes.py:BayesianLinear`):

$$q(w_{ij}) = \mathcal{N}\big(\mu_{ij},\, \sigma_{ij}^2\big), \qquad \sigma_{ij} = \mathrm{softplus}(\rho_{ij}) = \ln\big(1 + e^{\rho_{ij}}\big) > 0. \tag{5}$$

The learnable parameters are the pair (μ_{ij}, ρ_{ij}); ρ is used instead of σ directly so σ stays strictly positive without a constraint. A forward pass draws one sample per weight via the reparameterisation trick, which keeps the draw differentiable in (μ, ρ):

$$w_{ij} = \mu_{ij} + \sigma_{ij}\,\varepsilon_{ij}, \qquad \varepsilon_{ij} \sim \mathcal{N}(0,1) \qquad (\texttt{bayes.py:forward}). \tag{6}$$

The prior is a fixed zero-mean Gaussian $p(w_{ij}) = \mathcal{N}(0, \sigma_{\text{prior}}^2)$ with $\sigma_{\text{prior}} =$ `prior_sigma` (default 1.0).

§ 3 The ELBO objective

Variational inference maximises the Evidence Lower Bound over the whole dataset $\mathcal{D} = \{(X_i, y_i)\}_{i=1}^{N}$, $N \equiv N_{\text{train}}$:

$$\mathcal{L}_{\text{ELBO}}(q) = \underbrace{\mathbb{E}_{w\sim q}\Big[\textstyle\sum_{i=1}^{N} \ln p(y_i \mid X_i, w)\Big]}_{\text{expected data log-likelihood}} \;-\; \underbrace{\mathrm{KL}\big(q(w) \,\|\, p(w)\big)}_{\text{complexity penalty}}. \tag{7}$$

Maximising $\mathcal{L}_{\text{ELBO}}$ is equivalent to minimising the negative ELBO. CADDack minimises a per-example–scaled, minibatch estimate of it:

$$\boxed{\mathcal{L} = \underbrace{\mathrm{NLL}_{\text{batch}}}_{\text{data fit}} + \underbrace{\beta \cdot \frac{\mathrm{KL}_{\text{total}}}{N_{\text{train}}}}_{\text{regularisation}}} \qquad (\texttt{bayes.py:elbo\_loss}). \tag{8}$$

3.1 The data-fit term (heteroscedastic Gaussian NLL)

Assuming $y_i \sim \mathcal{N}\big(\mu_i,\, e^{s_i}\big)$, the negative log-likelihood of a batch of size B, dropping the constant $\frac{1}{2}\ln 2\pi$, is a *mean* over the batch:

$$\mathrm{NLL}_{\text{batch}} = \frac{1}{B}\sum_{i=1}^{B} \frac{1}{2}\Big[\, e^{-s_i}\,(y_i - \mu_i)^2 \;+\; s_i \,\Big]. \tag{9}$$

The e^{-s_i} factor down-weights noisy examples; the $+\,s_i$ term stops the model from declaring infinite noise. If `no_aleatoric=True`, this reduces to plain $\mathrm{NLL}_{\text{batch}} = \frac{1}{B}\sum_i (\mu_i - y_i)^2$ (MSE).

3.2 The complexity term (closed-form KL)

Because both q and p are Gaussian, the KL has a closed form per weight; summed over all Bayesian parameters (`bayes.py:kl`):

$$\mathrm{KL}_{\text{total}} = \sum_{\text{layers}} \sum_{i,j} \left[\ln \frac{\sigma_{\text{prior}}}{\sigma_{ij}} + \frac{\sigma_{ij}^2 + \mu_{ij}^2}{2\,\sigma_{\text{prior}}^2} - \frac{1}{2} \right]. \tag{10}$$

It is zero exactly when $q = p$ and grows as the posterior moves away from the prior; σ_{ij} is clamped to $\geq 10^{-12}$ before the log so the term can never diverge to $-\infty$.

3.3 Why divide the KL by N_{train}?

The dataset-level objective sums the NLL over all N examples but counts the KL *once*: $\sum_{i=1}^{N} \text{NLL}_i + \text{KL}$. Dividing the whole thing by N puts it on a per-example scale,

$$\frac{1}{N}\Big(\sum_{i=1}^{N} \text{NLL}_i + \text{KL}\Big) = \underbrace{\overline{\text{NLL}}}_{\text{a per-example mean}} + \frac{\text{KL}}{N}, \tag{11}$$

which is exactly Eq. (8) because $\text{NLL}_{\text{batch}}$ in Eq. (9) *is* a per-example mean. Hence the KL must be divided by the number of training examples N_{train}.

Common pitfall. Dividing instead by the number of minibatches $M = N/B$ (as an earlier version of the code did) leaves the NLL per-example but scales the KL by $\text{KL}/M = (B) \cdot \text{KL}/N$ — over-weighting the regulariser by a factor of the batch size B. The posterior then collapses toward the prior ($\mu_{ij} \to 0$), predictions regress to the mean, and epistemic uncertainty is inflated. CADDack divides by N_{train} to avoid this.

3.4 KL warmup (β annealing)

β ramps the KL in linearly over the first τ epochs (`train.py`):

$$\beta(\text{epoch}) = \beta_{\max} \cdot \min\Big(1,\ \tfrac{\text{epoch}+1}{\tau}\Big), \qquad \beta_{\max} = \texttt{kl\_weight},\ \ \tau = \texttt{kl\_warmup}. \tag{12}$$

Starting near $\beta \approx 0$ lets the means μ_{ij} acquire signal before the regulariser pulls $\sigma_{ij} \to \sigma_{\text{prior}}$; without warmup the network can get stuck at the prior.

§ 4 How it is minimised

The loss $\mathcal{L}$ in Eq. (8) is minimised by Adam. Each step uses *one* weight sample (an unbiased Monte-Carlo estimate of $\mathbb{E}_q[\text{NLL}]$), while the KL is exact — no sampling. Gradients flow to (μ_{ij}, ρ_{ij}) through the reparameterised w_{ij} and to the deterministic towers/heads normally.

```
n_train = number of training complexes        # N
opt     = Adam(model.parameters(), lr)
for epoch in range(epochs):
    beta = kl_weight * min(1, (epoch+1)/kl_warmup)   # KL annealing
    for (lig_batch, geo_batch) in train_loader:      # two-graph batch
        opt.zero_grad()
        mu, log_var = model(lig_batch, geo_batch)    # one weight sample
        y           = geo_batch.y
        kl          = model.kl()                     # closed form, Eq.10
        loss = NLL_batch(mu, log_var, y) + beta*kl/n_train   # Eq.8
        loss.backward()                              # grad wrt mu, rho
        opt.step()
```

This is the training loop of `train.py:train_fusion_from_complexes`. After training, prediction draws T weight samples and splits the predictive variance into epistemic ($\text{Var}_t \hat{\mu}_t$) and aleatoric ($\mathbb{E}_t\, e^{s_t}$) parts (`models.py:predict_with_uncertainty`); the biased variance estimator is used so a single sample yields 0 rather than `NaN`.

Gradient of the reparameterised weight (used implicitly by autograd):

$$\frac{\partial w_{ij}}{\partial \mu_{ij}} = 1, \qquad \frac{\partial w_{ij}}{\partial \rho_{ij}} = \varepsilon_{ij}\, \frac{d}{d\rho}\text{softplus}(\rho_{ij}) = \frac{\varepsilon_{ij}}{1 + e^{-\rho_{ij}}}. \tag{13}$$

§ 5 Symbol / code / shape cross-reference

Math	Code	Shape	Where
μ	`mu`	$[B]$	`bayes.py:BayesianMLP.forward`
$s = \ln \sigma_{\text{al}}^2$	`log_var`	$[B]$	`bayes.py:BayesianMLP.forward`
y	`geo_batch.y`	$[B]$	`train.py`
KL_{total}	`model.kl()`	scalar	`bayes.py:kl`
N_{train}	`n_train`	scalar	`train.py:272`
β	`beta`	scalar	`train.py:303`
$\mathcal{L}$	`loss`	scalar	`bayes.py:elbo_loss`
(μ_{ij}, ρ_{ij})	`weight_mu, weight_rho`	layer	`bayes.py:BayesianLinear`
σ_{prior}	`prior_sigma`	scalar	`bayes.py:BayesianLinear`